

Computer Vision Lab 04: Exhaustive Exam Master Cheat-Sheet

Course Code: AI-4002 | Target Manual: Lab 04 Manual (Feature Extraction, Filtering, Convolution & Edge Detection)

Institution: NUCES Karachi | Instructor: Talha Shahid

1. Core Theoretical Foundations

1.1 Introduction to Feature Extraction

In computer vision, **Feature Extraction** is a fundamental preprocessing and data representation technique. It is defined as the process of selecting, computing, and transforming relevant information or visual patterns from raw visual data (such as digital images or video frames) to construct a compact, lower-dimensional, and semantically meaningful representation.

- **Raw Visual Data:** Computer vision workflows begin by capturing raw images or video streams. These raw pixel matrices contain an immense volume of individual intensity values, color channels, and spatial textures.
- **Need for Feature Extraction:** Raw visual data is excessively high-dimensional, computationally prohibitive, and sensitive to trivial environmental noise. Direct analysis on raw pixel grids leads to severe computational bottlenecks and model overfitting. Feature extraction reduces dimensionality, isolates essential structural attributes, and simplifies downstream classification, recognition, or alignment tasks.
- **What is a Feature?** A feature is a distinctive, recognizable attribute, characteristic, or pattern extracted from visual data. A high-quality feature captures essential structural properties (e.g., corners, edges, textures, color distributions) while discarding irrelevant background noise and redundant information.

1.2 Local Features vs. Global Features

Feature extraction techniques are broadly categorized based on their spatial scope across the image plane:

Feature Type	Definition & Scope	Primary Use Cases	Key Examples
Local Features	Extracted from specific, localized regions, keypoints, or small image patches.	Image matching, object alignment, 3D reconstruction, tracking under partial occlusion.	Keypoint descriptors, Harris corners, SIFT/SURF patches, local LBP masks.
Global Features	Computed across the entire image area as a single unified global descriptor vector.	Overall scene classification, content-based image retrieval, material/texture classification.	Color histograms, global texture descriptors, image moments, full-image LBP distributions.

1.3 Feature Extraction Approaches & Real-World Applications

- **Histogram-Based Methods:** Capture statistical frequency distributions of visual properties (color intensities, gradient orientations, edge directions, local binary texture patterns) across local regions or global images.
- **Filter-Based Methods:** Convolve specialized mathematical filter banks (e.g., Gabor filters, Haar wavelets, Sobel operators) across the image plane to detect specific directional edges, ripples, or spatial frequencies.
- **Deep Learning Features:** Convolutional Neural Networks (CNNs) automatically learn hierarchical spatial features directly from raw pixel matrices across stacked convolutional and pooling layers.

- **Real-World Applications:** Object Recognition (pedestrians, vehicles), Face Verification, Gesture Recognition (human-computer interaction), Medical Image Diagnostics (tumor/anomaly detection in MRI/CT scans), Autonomous Vehicle Perception (obstacle and lane detection).
- **Real-World Challenges:** Lighting/illumination variations, scale differences, rotation/orientation shifts, background clutter, and sensor noise. Choosing appropriate features and normalization parameters is critical.

2. Detailed Histogram-Based Feature Extraction Methods

2.1 Histogram of Oriented Gradients (HOG)

HOG is a widely used feature descriptor that models local object shape and structure by capturing the statistical distribution of intensity gradient directions across localized spatial regions. It is especially effective for pedestrian and object detection.

Step-by-Step Working Mechanism of HOG:

- **1. Image Preprocessing:** Convert the input image to grayscale to simplify gradient calculations.
- **2. Gradient Computation:** Calculate the horizontal (dx) and vertical (dy) gradients at each pixel using Sobel operators. Compute gradient magnitude $G = \sqrt{dx^2 + dy^2}$ and orientation angle $\theta = \arctan2(dy, dx)$.
- **3. Cell Division:** Divide the image into small, non-overlapping spatial grids called *cells* (e.g., 8x8 pixels per cell).
- **4. Orientation Histograms:** For each cell, construct a 9-bin orientation histogram (covering 0° to 180° or 0° to 360°). Each pixel within the cell casts a magnitude-weighted vote into its corresponding angle bin.
- **5. Block Normalization:** Group adjacent cells into larger overlapping *blocks* (e.g., 2x2 cells = 16x16 pixels). Compute the L2-norm across the block and normalize cell histograms to mitigate illumination and shadow variations.
- **6. Feature Vector Assembly:** Concatenate all normalized block histograms into a single 1D feature vector.

Exam-Ready Python Code — HOG Extraction (skimage & OpenCV):

```

import cv2
import matplotlib.pyplot as plt
from skimage.feature import hog
from skimage import exposure
# 1. Read input image in Grayscale mode
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
# 2. Extract HOG feature vector and visualization image using skimage
features, hog_image = hog(
    image,
    orientations=9,          # 9 orientation bins (0 to 180 degrees)
    pixels_per_cell=(8, 8),  # Cell dimensions: 8x8 pixels per cell
    cells_per_block=(2, 2),  # Block dimensions: 2x2 cells for contrast normalization
    visualize=True           # Return 2D visualization array
)
# 3. Rescale visualization intensity for clear display
hog_image_rescaled = exposure.rescale_intensity(hog_image, in_range=(0, 10))
# 4. Display Original vs HOG Visualization
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Grayscale Image')
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(hog_image_rescaled, cmap='gray')
plt.title('HOG Feature Visualization')
plt.axis('off')
plt.show()
print("HOG Feature Vector Length:", len(features))

```

2.2 Local Binary Pattern (LBP)

LBP is an efficient texture feature descriptor that captures local micro-patterns by comparing the intensity of each central pixel with its surrounding circular neighborhood pixels.

Step-by-Step Working Mechanism of LBP:

- **1. Image Preprocessing:** Convert image to grayscale.
- **2. Neighborhood Definition:** Select a circular neighborhood of radius R with P sampling points around center pixel i_c .
- **3. Intensity Thresholding:** Compare each neighbor intensity i_p with center i_c . Assign binary value 1 if $i_p \geq i_c$, else assign 0.
- **4. Pattern Formation:** Concatenate the binary comparison results clockwise to form a P-bit binary number and convert to decimal.
- **5. Histogram Calculation:** Count the frequency occurrences of each LBP decimal code across the image to build a texture distribution histogram.
- **6. Feature Vector:** The normalized LBP histogram serves as the final feature vector for texture classification and face verification.

Exam-Ready Python Code — LBP Texture Descriptor:

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import local_binary_pattern
# 1. Read input image in Grayscale
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
# 2. Define circular neighborhood parameters
radius = 1 # Neighborhood radius in pixels
n_points = 8 * radius # Number of circular sampling points
# 3. Compute LBP code matrix using uniform pattern method
lbp_image = local_binary_pattern(image, n_points, radius, method='uniform')
# 4. Calculate normalized LBP histogram
n_bins = int(lbp_image.max() + 1)
hist, _ = np.histogram(lbp_image.ravel(), bins=n_bins, range=(0, n_bins))
hist = hist.astype("float")
hist /= (hist.sum() + 1e-6) # Normalize histogram
# 5. Display LBP image and feature histogram
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(lbp_image, cmap='gray')
plt.title('LBP Texture Map')
plt.axis('off')
plt.subplot(1, 2, 2)
plt.bar(range(n_bins), hist)
plt.title('Normalized LBP Histogram')
plt.xlabel('LBP Pattern Index')
plt.ylabel('Frequency')
plt.show()
print("LBP Feature Vector:", hist)

```

2.3 Histogram of Color

Captures the global or regional distribution of pixel color intensities across an image by dividing color space (RGB, HSV, LAB) into discrete bins and counting pixel frequencies.

```

import cv2
import matplotlib.pyplot as plt
# Read image in standard BGR color format
image = cv2.imread('image.jpg')
# Calculate color histogram for each channel (B, G, R)
colors = ('b', 'g', 'r')
plt.figure(figsize=(8, 4))
for i, col in enumerate(colors):
    # cv2.calcHist([images], [channels], mask, [histSize], [ranges])
    hist = cv2.calcHist([image], [i], None, [256], [0, 256])
    plt.plot(hist, color=col)
    plt.xlim([0, 256])
plt.title('Color Histogram (RGB/BGR Channels)')
plt.xlabel('Pixel Intensity (0-255)')
plt.ylabel('Pixel Count')
plt.show()

```

2.4 Histogram of Edge Directions (HED)

HED isolates structural geometry by evaluating gradient orientation angles **ONLY at detected edge pixel locations** (e.g., Canny edge map), ignoring uniform background areas.

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
# 1. Load image in grayscale and apply Gaussian blur
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
image_smoothed = cv2.GaussianBlur(image, (5, 5), 0)
# 2. Perform Canny edge detection to obtain binary edge map
edges = cv2.Canny(image_smoothed, threshold1=30, threshold2=70)
# 3. Calculate horizontal (dx) and vertical (dy) gradients
gradient_x = cv2.Sobel(image_smoothed, cv2.CV_64F, 1, 0, ksize=3)
gradient_y = cv2.Sobel(image_smoothed, cv2.CV_64F, 0, 1, ksize=3)
# 4. Compute gradient orientation in degrees (0 - 360)
gradient_orientation = np.arctan2(gradient_y, gradient_x) * (180.0 / np.pi) % 360.0
# 5. Extract orientations ONLY at detected edge pixel coordinates
edge_orientations = gradient_orientation[edges > 0]
# 6. Bin orientations into 8 direction bins (octagonal ranges)
hist, bin_edges = np.histogram(edge_orientations, bins=8, range=(0, 360))
# 7. Plot Edge Map and HED Histogram
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(edges, cmap='gray')
plt.title('Canny Edge Map')
plt.axis('off')
plt.subplot(1, 2, 2)
plt.bar(bin_edges[:-1], hist, width=35, align='center')
plt.title('Histogram of Edge Directions (HED)')
plt.xlabel('Edge Orientation (Degrees)')
plt.ylabel('Frequency')
plt.show()
print("HED Feature Vector:", hist)

```

2.5 Histogram of Intensity Gradients (HIG)

HIG evaluates gradient magnitudes and orientation angles across **ALL pixels in the entire image**. The orientation histogram is weighted by gradient magnitude to emphasize strong structural transitions.

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
# 1. Load image in grayscale
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
# 2. Compute Horizontal (dx) and Vertical (dy) Gradients using Sobel
dx = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
dy = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)
# 3. Compute Magnitude and Orientation (0 - 180 degrees)
magnitude = np.sqrt(dx**2 + dy**2)
orientation = np.arctan2(dy, dx) * (180.0 / np.pi) % 180.0
# 4. Construct 9-bin orientation histogram weighted by gradient magnitude across ALL pixels
histogram, bins = np.histogram(orientation, bins=9, range=(0, 180), weights=magnitude)
# 5. Normalize feature vector
histogram_norm = histogram / (histogram.sum() + 1e-6)
# 6. Display HIG Histogram
plt.figure(figsize=(6, 4))
plt.bar(bins[:-1], histogram_norm, width=15, align='edge')
plt.title('Histogram of Intensity Gradients (HIG)')
plt.xlabel('Gradient Orientation (Degrees)')
plt.ylabel('Normalized Frequency')
plt.show()
print("HIG Feature Vector:", histogram_norm)

```

2.6 Texture Energy and Contrast Histograms

- **Texture Energy:** Measures local uniformity or smoothness within an analysis window (3x3 or 5x5). Calculated as the sum of squared pixel intensities: $E = \sum(\text{pixel_values}^2)$. Higher energy indicates complex, rough, non-uniform texture.
- **Texture Contrast:** Quantifies local intensity variation or tonal difference within an analysis window. Calculated as the standard deviation of pixel intensities: $C = \sqrt{\text{Var}(\text{pixel_values})}$. Higher contrast indicates sharp local variations.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# 1. Load image in grayscale and convert to float32
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE).astype(np.float32)
# 2. Define neighborhood window size (3x3)
window_size = 3
kernel = np.ones((window_size, window_size), dtype=np.float32)
# 3. Calculate Texture Energy: Local sum of squared pixel values via filter2D
energy_map = cv2.filter2D(image ** 2, -1, kernel)
# 4. Calculate Texture Contrast: Local standard deviation
mean_map = cv2.filter2D(image, -1, kernel) / (window_size ** 2)
mean_sq_map = cv2.filter2D(image ** 2, -1, kernel) / (window_size ** 2)
variance_map = np.maximum(0, mean_sq_map - mean_map ** 2)
contrast_map = np.sqrt(variance_map)
# 5. Compute 256-bin Feature Histograms
energy_hist, energy_bins = np.histogram(energy_map.ravel(), bins=256, range=(0, energy_map.max()))
contrast_hist, contrast_bins = np.histogram(contrast_map.ravel(), bins=256, range=(0, contrast_map.max()))
# 6. Normalize Histograms
energy_hist = energy_hist / (energy_hist.sum() + 1e-6)
contrast_hist = contrast_hist / (contrast_hist.sum() + 1e-6)
# 7. Plot Texture Histograms
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(energy_bins[:-1], energy_hist, color='b')
plt.title('Texture Energy Histogram')
plt.xlabel('Energy')
plt.subplot(1, 2, 2)
plt.plot(contrast_bins[:-1], contrast_hist, color='r')
plt.title('Texture Contrast Histogram')
plt.xlabel('Contrast')
plt.tight_layout()
plt.show()
```

3. Spatial Filtering & Convolution Mechanics

3.1 Filtering vs. Convolution Definitions

Filtering: Involves applying a small numerical matrix (filter or kernel) across an input image to modify pixel values, enhancing useful structural features or suppressing noise.

Convolution: The mathematical operation used to perform filtering. It involves sliding the kernel across the image, computing element-wise dot products between kernel weights and underlying pixel values, and summing the results.

3.2 Kernel Cookbook & Filter Operators

Common filter kernels and their operations:

Filter Type	Kernel Matrix	Purpose & Visual Effect
Box Blur	$1/9 * \begin{bmatrix} 1, 1, 1 \\ 1, 1, 1 \\ 1, 1, 1 \end{bmatrix}$	Simple averaging filter. Reduces noise but causes blurring.
Gaussian Blur	Calculated via <code>cv2.getGaussianKernel(ksize, sigma)</code>	Weighted spatial smoothing. Suppresses noise while preserving edges better than Box Blur.
Sobel Horizontal (dx)	$\begin{bmatrix} -1, 0, 1 \\ -2, 0, 2 \\ -1, 0, 1 \end{bmatrix}$	1st derivative gradient filter. Detects vertical edges.
Sobel Vertical (dy)	$\begin{bmatrix} -1, -2, -1 \\ 0, 0, 0 \\ 1, 2, 1 \end{bmatrix}$	1st derivative gradient filter. Detects horizontal edges.
Scharr Horizontal (dx)	$\begin{bmatrix} -3, 0, 3 \\ -10, 0, 10 \\ -3, 0, 3 \end{bmatrix}$	High-accuracy gradient operator. Superior angle accuracy over Sobel.
Embossing	$\begin{bmatrix} -2, -1, 0 \\ -1, 1, 1 \\ 0, 1, 2 \end{bmatrix}$	Emphasizes local intensity differences to create a 3D shadowed relief effect.

3.3 The 4 Convolution Types (Exam Critical)

- **1. Standard / Full Convolution:** Centers the kernel over every pixel. Output includes boundary overlap regions.
- **2. Valid Convolution (No Padding):** Only processes pixels where the kernel fully overlaps the input image without crossing boundaries. *Output dimensions shrink to $(H - k_h + 1) \times (W - k_w + 1)$.*
- **3. Same Convolution (Padding Added):** Adds zero or reflection padding around image borders so output dimensions match input dimensions ($H \times W$).
- **4. Valid Convolution with Strides:** Moves the kernel across the image by step size $s > 1$ (e.g., stride = 2), downsampling output dimensions to $\text{floor}((H - k)/s) + 1$.

Exam-Ready Python Code — All 4 Convolution Types:

```
import cv2
import numpy as np
# Load grayscale image
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
# Define 3x3 Sobel Horizontal kernel
kernel = np.array([[-1, 0, 1],
                  [-2, 0, 2],
                  [-1, 0, 1]], dtype=np.float32)
# 1. Standard / Same Convolution (Reflect Padding -> Output shape = Input shape)
same_conv = cv2.filter2D(image, -1, kernel, borderType=cv2.BORDER_REFLECT)
# 2. Valid Convolution (No Padding -> Manual crop to valid kernel overlap area)
# BorderConstant padding creates 0-border, then slice removes padded boundary
raw_valid = cv2.filter2D(image, -1, kernel, borderType=cv2.BORDER_CONSTANT)
kh, kw = kernel.shape
valid_conv = raw_valid[kh//2 : -(kh//2), kw//2 : -(kw//2)]
# 3. Same Convolution with Zero Padding
same_zero_pad = cv2.filter2D(image, -1, kernel, borderType=cv2.BORDER_CONSTANT)
# 4. Valid Convolution with Strides (Stride = 2)
# Apply valid convolution then step by stride factor 2
strided_conv = valid_conv[::2, ::2]
print("Original Image Shape:", image.shape)
print("Same Conv Shape:", same_conv.shape)
print("Valid Conv Shape:", valid_conv.shape)
print("Strided Conv (stride=2) Shape:", strided_conv.shape)
```

4. Edge Detection & Common Techniques

4.1 Overview of Edge Detection Operators

- **Gradient-Based Operators (Sobel, Prewitt, Scharr):** Compute 1st-order derivatives to locate peaks in intensity rate of change.
- **Laplacian of Gaussian (LoG) / Marr-Hildreth:** Combines Gaussian smoothing with 2nd-order Laplacian operator. Locates edges by finding *zero-crossings*.
- **Sobel-Feldman Operator:** Variation of Sobel emphasizing diagonal edges in addition to horizontal/vertical.
- **CNN-Based Edge Detection:** Deep learning networks that learn multi-scale visual edge features directly from data.

Gradient-Based Edge Detection Code Example:

```
import cv2
import numpy as np
# Load image
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
# Calculate gradients along X and Y axes
gradient_x = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
gradient_y = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)
# Compute gradient magnitude and direction
gradient_magnitude = np.sqrt(gradient_x**2 + gradient_y**2)
gradient_direction = np.arctan2(gradient_y, gradient_x)
# Threshold gradient magnitude to create binary edge map
threshold = 100
edges = (gradient_magnitude > threshold).astype(np.uint8) * 255
```

4.2 The Canny Edge Detector (5-Stage Algorithm)

Developed by John F. Canny in 1986, this is the optimal multi-stage edge detection algorithm.

- **1. Gaussian Smoothing:** Convolves the image with a Gaussian kernel to reduce noise.
- **2. Gradient Calculation:** Applies Sobel masks to compute magnitude G and orientation angle θ , rounded to 0° , 45° , 90° , or 135° .
- **3. Non-Maximum Suppression (NMS):** Edge thinning. Suppresses non-peak gradient magnitudes along the gradient direction line.
- **4. Double Thresholding:** Classifies pixels using high (T_{high}) and low (T_{low}) thresholds into Strong edges ($\geq T_{\text{high}}$), Weak edges (T_{low} to T_{high}), and Non-edges ($< T_{\text{low}}$).
- **5. Edge Tracking by Hysteresis:** Weak edges are retained ONLY if spatially connected to a Strong edge pixel.

Exam-Ready Canny Edge Detection Code:

```
import cv2
# Read image in grayscale
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
# Apply Gaussian blur for noise suppression
blurred = cv2.GaussianBlur(image, (5, 5), sigmaX=1.0)
# Apply Canny Edge Detector (T_low = 50, T_high = 150)
canny_edges = cv2.Canny(blurred, threshold1=50, threshold2=150, apertureSize=3)
cv2.imshow('Canny Edges', canny_edges)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

5. Quick Reference & Exam Survival Kit

5.1 Feature Descriptors Comparison Matrix

Feature	Objective	Input / Preprocessing	Key Output / Vector
HOG	Object & Pedestrian Detection	Grayscale, 8x8 cells, 2x2 blocks	Normalized 1D gradient orientation histogram
LBP	Texture & Face Recognition	Grayscale, circular neighborhood (R, P)	Decimal LBP code frequency histogram
HED	Edge Direction Distribution	Grayscale -> Canny/Sobel Edge Map	Histogram of edge gradient angles (0-360°)
HIG	Global Gradient Map	Grayscale image (ALL pixels)	Histogram of gradient angles weighted by magnitude
Texture Energy	Local Smoothness / Uniformity	Grayscale, 3x3 window	Sum of squared intensities in window
Texture Contrast	Local Intensity Variance	Grayscale, 3x3 window	Standard deviation of intensities in window

5.2 MCQ Traps & Exam Trick Alerts

EXAM TRICK ALERT

Trap 1 (HOG Block Normalization): HOG normalizes across overlapping blocks to make descriptors invariant to illumination and shadow variations.

EXAM TRICK ALERT

Trap 2 (LBP Invariance): LBP is invariant to monotonic grayscale transformations (brightness shifts), but NOT rotation invariant unless uniform rotation-invariant LBP is specified.

EXAM TRICK ALERT

Trap 3 (HED vs. HIG): HED evaluates orientation angles **ONLY at edge pixels**. HIG evaluates orientation angles across **ALL image pixels**.

EXAM TRICK ALERT

Trap 4 (Valid Convolution Output Dimensions): A 5x5 kernel applied with Valid Convolution to a 100x100 image yields size $(100 - 5 + 1) = 96 \times 96$, NOT 100x100!

EXAM TRICK ALERT

Trap 5 (Canny Hysteresis Rule): Weak edges (between T_low and T_high) are preserved **ONLY** if spatially connected to a Strong edge.

EXAM TRICK ALERT

Trap 6 (LoG vs Sobel Derivatives): Sobel computes 1st derivatives (gradient peaks). LoG computes 2nd derivatives (zero-crossings).

5.3 WHICH FUNCTION DO I USE?

Exam Task Requirement	Primary Function / API	Key Parameters
Extract HOG Features & Visualization	<code>skimage.feature.hog()</code>	<code>orientations=9, pixels_per_cell=(8,8), cells_per_block=(2,2), visualize=True</code>
Extract LBP Texture Image	<code>skimage.feature.local_binary_pattern()</code>	<code>image, P=n_points, R=radius, method='uniform'</code>
Apply Custom Kernel / Filter	<code>cv2.filter2D()</code>	<code>src, ddepth=-1, kernel, borderType=cv2.BORDER_REFLECT</code>
Compute Image Gradients (Sobel)	<code>cv2.Sobel()</code>	<code>src, ddepth=cv2.CV_64F, dx=1, dy=0, ksize=3</code>
Apply Canny Edge Detector	<code>cv2.Canny()</code>	<code>image, threshold1=T_low, threshold2=T_high, apertureSize=3</code>

5.4 Common Coding Errors & Fixes

Observed Error	Root Cause	Exam Fix / Solution
Black output after <code>cv2.Sobel()</code>	Negative gradient values clipped when saved directly as <code>uint8</code> .	Use <code>ddepth=cv2.CV_64F</code> , then <code>cv2.convertScaleAbs()</code> or <code>np.absolute()</code> .
<code>AttributeError: 'cv2' has no 'hog'</code>	HOG is in <code>skimage</code> , not <code>OpenCV</code> .	Import via <code>from skimage.feature import hog</code> .
Canny output completely black/white	Thresholds <code>T_low</code> and <code>T_high</code> set incorrectly.	Adjust thresholds (e.g., <code>T_low=50, T_high=150</code>) or apply <code>GaussianBlur</code> first.